

Counterfactual Full-Action Bellman Supervision for Temporally Coupled Satellite-Ground Scheduling

Bocheng Yang¹, Lujie Zheng¹, Jian Liu^{1,2,*}, and Bo Chen^{1,*}

¹School of Aerospace, Harbin Institute of Technology (Shenzhen), Shenzhen 518055, China

²Guangxi Key Laboratory of Precision Navigation Technology and Application,
Guilin University of Electronic Technology, Guilin 541004, China

*Correspondence: liujian2024@hit.edu.cn; hitchenbo@hit.edu.cn

Abstract

Satellite-ground scheduling is temporally coupled because a locally inexpensive action can consume resources needed by later tasks. A standard deep Q-network (DQN) trains only the executed action even when an analytical model can evaluate every one-step alternative. We introduce MD-CBAD-DQN, which constructs Bellman targets for all three processing modes and distills their centered within-state advantages. Training therefore evaluates the two unchosen modes, whereas deployment uses no preview. The factual target and loss term are retained, while the auxiliary gradient changes the total update. We evaluate a fully specified synthetic, soft-constrained single-satellite simulator using 20 model seeds and shared held-out traces. Against standard DQN, MD-CBAD-DQN reduced mean composite cost by 0.87%–1.72% across five fully coupled regimes under the prespecified paired common-random-number analysis. Uncentered full-action Bellman supervision matched CBAD within uncertainty, whereas removing the bootstrap weakened the benefit. Double-CBAD also had lower nominal mean cost than Double DQN, although its family-adjusted sign test did not cross 0.05. Exact planning reduced mean cost from 99.33 at one step to 89.35 at six steps while increasing mean action-selection time from 0.13 to 41.60 ms. Both principal learned policies reached zero remaining energy in some episodes, so the results establish neither hard-constrained deployability nor validity for flight dynamics outside the defined simulator.

Keywords: satellite edge computing; task scheduling; model-driven reinforcement learning; counterfactual supervision; Bellman advantage distillation.

1 Introduction

Earth-observation and communication satellites increasingly execute perception and decision workloads in orbit. Compact networks can reduce downlink demand and response latency under constrained onboard resources [1–5]. Yet onboard execution consumes heat, energy, and compute capacity, while ground offloading consumes contact time and bandwidth. A scheduler must therefore choose among onboard execution, complete offloading, and collaborative preprocessing.

Existing schedulers optimize constellation coordination, observation plans, service placement, deadlines, makespan, or energy use [6–12]. The present decision instead changes six resources that persist across task arrivals. These resources are heat, energy, queue occupancy, compute utilization, bandwidth, and remaining contact. Consider two consecutive tasks. Onboard execution may be cheapest for the first task, yet its heat and energy use can make ground or collaborative processing preferable when the second task arrives. The scientific question is therefore whether model-defined one-step alternatives can teach a policy these action gaps without requiring preview at deployment.

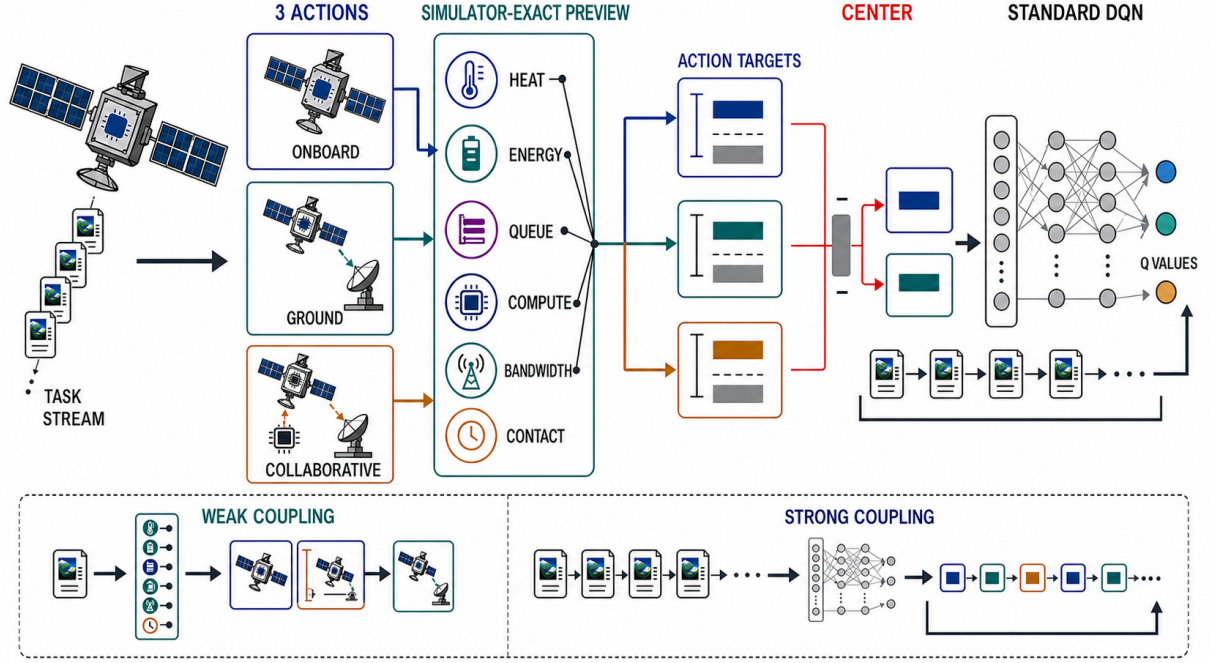


Figure 1: Graphical overview of MD-CBAD-DQN. Simulator-exact analytical previews expose the immediate cost and six post-action resources for all three scheduling actions. Centering the resulting Bellman targets isolates action-relative supervision. The factual standard-DQN target and loss term are retained, and the auxiliary loss contributes an additional gradient during training. The illustration is conceptual and encodes no quantitative results.

Standard DQN [13, 14] learns one Bellman target from each factual tuple (s_t, a_t, r_t, s_{t+1}) . Here, the resource equations can also evaluate every unexecuted action without advancing the environment or revealing later tasks. The unresolved question is how to use these simulator-exact alternatives while preserving a controlled standard-DQN comparison. Absolute counterfactual targets provide more supervision, but their common state-value offset does not affect action selection.

We address this question with MD-CBAD-DQN. The method previews every action, forms standard target-network Bellman targets, and centers predicted and target values within each state. It then distills the resulting action advantages while the factual Huber loss anchors the absolute Q scale. Counterfactual previews are restricted to training.

Our contributions are:

1. We formulate three satellite-ground processing modes as a 21-dimensional sequential problem and derive coupled heat, energy, latency, and transmission descriptors from common task primitives.
2. We introduce a counterfactual full-action Bellman objective whose centered implementation supervises the action gaps that determine scheduling decisions.
3. We evaluate this objective against a strictly matched standard-DQN backbone. Component controls support Bellman-supervised full-action labels but do not isolate centering as necessary. Coupling controls, a Double-DQN pair, and finite-horizon planners further bound the result.

2 Related Work and Novelty Positioning

2.1 Satellite-ground scheduling

Satellite schedulers cover constellation coordination, observation planning, service placement, and energy-aware offloading [6–12]. Our setting combines an explicit cost for every action with resource carryover between task arrivals. The one-step objective is therefore observed rather than hidden. Learning is justified only when present actions change future feasibility.

2.2 Model use and counterfactual learning

Dyna established predicted experience for value learning and planning [15], and model-based acceleration has also been used to supplement deep Q-learning with locally fitted dynamics [16]. Counterfactual data fusion and causal augmentation construct transitions under structural assumptions [17, 18]. Counterfactual credit assignment instead separates action influence from future randomness [19]. Recent work also studies multi-action generative interfaces and joint quantities across action outcomes [20]. Counterfactual information and model-assisted Q-learning are therefore not new by themselves.

Advantage learning widens action gaps through modified operators [21], whereas MD-CBAD-DQN retains the chosen DQN Bellman operator and adds a supervised action-gap loss. Unlike Dyna-style replay synthesis, the method inserts no modeled transition into replay. Unlike model-based acceleration through local rollouts, it uses one analytical outcome for every discrete action only inside an auxiliary target. The component study below directly compares centered Bellman advantages with uncentered full-action Q targets and centered immediate-cost targets. Our novelty claim is consequently narrow. We claim the evaluated full-action Bellman auxiliary design and its scheduling application, not counterfactual simulation, model-assisted Q-learning, advantage learning, or centering in general.

3 System Model and Immediate-Cost Quantification

At task arrival t , a satellite selects one of three actions:

$$\mathcal{A} = \{\alpha_s, \alpha_g, \alpha_{s\&g}\}, \quad (1)$$

where α_s executes onboard and downlinks the result, α_g downlinks the complete input for ground execution, and $\alpha_{s\&g}$ performs onboard preprocessing followed by ground execution.

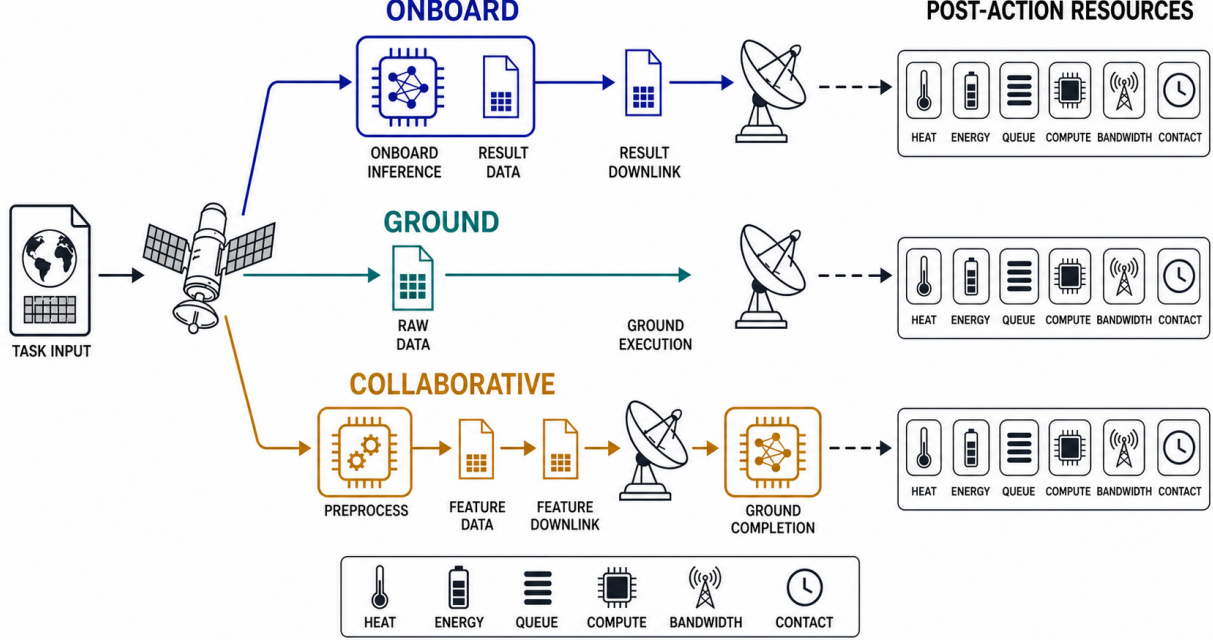


Figure 2: Three satellite-ground processing pathways. Each arriving task is executed onboard, transferred for ground execution, or preprocessed collaboratively before ground completion. The analytical transition model returns the immediate cost and the post-action heat, energy, queue occupancy, compute utilization, bandwidth, and remaining contact.

3.1 Implemented immediate-cost model

The revised experiments use the dimensionless linear cost implemented by the physical-consistency simulator. Let x_k denote the 15 task descriptors in Table 1. Each descriptor is normalized as

$$\bar{x}_k = \text{clip}(x_k/s_k, 0, 1.5), \quad (2)$$

where s_k is the corresponding scale. For inherited normalized bandwidth b , the available return rate is $R(b) = 2.2 + 217.8b \text{ Mbit s}^{-1}$. The three action latencies are

$$\begin{aligned} L_s &= x_3 + 1000x_6/R(b), \\ L_g &= x_8 + 1000x_9/R(b), \\ L_{s\&g} &= 1000x_{12}/4 + x_{13} + 1000x_{15}/R(b). \end{aligned} \quad (3)$$

The implemented immediate costs are

$$\begin{aligned}
c_s^{imm} &= 0.70\bar{x}_1 + 0.65\bar{x}_2 + 0.00045L_s + 0.55\bar{x}_5 + 0.35\bar{x}_6 + 0.35h + 0.12(1 - e), \\
c_g^{imm} &= 0.45\bar{x}_7 + 0.00045L_g + 0.55\bar{x}_9 + 0.35(0.28 - \tau)_+, \\
c_{s\&g}^{imm} &= 0.55(\bar{x}_{10} + \bar{x}_{11}) + 0.55\bar{x}_{14} + 0.00045L_{s\&g} + 0.45\bar{x}_{15} \\
&\quad + 0.18h + 0.12q + 0.18(0.22 - \tau)_+.
\end{aligned} \tag{4}$$

Here h , e , q , and τ are inherited heat, remaining energy, queue occupancy, and contact duration. The weights are fixed simulator assumptions rather than fitted mission utilities. They are applied identically during training, counterfactual preview, MPC evaluation, and deployment evaluation. Automated tests reproduce Eq. (4) from the 15 raw descriptors and compare it with all three code outputs.

Table 1: Implemented 15-dimensional task descriptor dictionary. The six appended resource coordinates are h , e , q , compute utilization u , bandwidth b , and contact duration τ .

Coordinate	Descriptor	Unit	Normalization scale s_k
x_1	Onboard compute heat	J	100
x_2	Total workload	GFLOP	4.8
x_3	Onboard compute time	ms	1400
x_4	Result transmit time at 50 Mbit s ⁻¹	ms	100
x_5	Squared co-location count	count ²	2401
x_6	Onboard-result data	Mbit	4
x_7	Full-offload transmit heat	J	100
x_8	Ground compute time	ms	100
x_9	Raw input data	Mbit	76.8
x_{10}	Preprocessing heat	J	100
x_{11}	Feature-transmit heat	J	100
x_{12}	Onboard preprocessing work	GFLOP	2.2
x_{13}	Remaining ground compute time	ms	100
x_{14}	Total workload descriptor	GFLOP	4.8
x_{15}	Collaborative feature data	Mbit	32

3.2 Physically coupled task and communication generator

The revised simulator does not sample heat, workload, data volume, and latency independently. It first draws correlated latent variables (v_t, w_t) with correlation parameter 0.68 and maps them through a logistic transform to raw data volume $D_t \in [8, 64]$ Mbit and workload $W_t \in [0.25, 4.0]$ GFLOP. Result and collaborative-feature volumes are

$$D_t^{res} = r_t D_t, \quad r_t \in [0.01, 0.05], \quad D_t^{feat} = f_t D_t, \quad f_t \in [0.08, 0.30]. \tag{5}$$

The implementation enforces $D_t^{res} \leq D_t^{feat} \leq D_t$. A workload-dependent fraction $p_t \in [0.20, 0.45]$ is processed onboard in the collaborative mode. Given onboard and ground rates $R_s = 4$ and $R_g = 80$ GFLOP s⁻¹, processing time is derived as W/R , rather than sampled separately. Compute power

risers monotonically from 4 to 20 W with workload; compute heat is 0.78 times compute energy. Transmission time is data volume divided by the effective return rate

$$R_t^{link} = 2.2 + 217.8b_t \quad \text{Mbit s}^{-1}, \quad (6)$$

and transmission heat is 0.55 times transmit energy. These values are engineering envelopes, not fitted flight distributions. Their scale is consistent with the range of modern SmallSat computing-system power reported by NASA and with NASA ground-network examples spanning 2.2 Mbit s⁻¹ S-band to 220 Mbit s⁻¹ X-band returns [22, 23]. The bounded data-reduction factors represent the bandwidth-reduction role of onboard image compression, not a claim about one instrument-specific codec [24]. Table 2 states every implemented assumption.

Table 2: Physical primitives and provenance of the revised simulator. Values define a transparent engineering envelope; they are not presented as a fitted flight distribution.

Quantity	Implemented range	Basis
Raw task data	8–64 Mbit nominal	Stress interval reaches 92.8 Mbit before sensitivity scaling
Arithmetic workload	0.25–4.0 GFLOP nominal	Stress interval reaches 5.8 GFLOP
Result / raw-data ratio	1–5%	Bounded compression assumption
Feature / raw-data ratio	8–30%	Bounded collaborative-processing assumption
Preprocessing fraction	20–45%	Deterministic function of workload quantile
Onboard / ground rate	4 / 80 GFLOP s ⁻¹	Fixed simulator hardware envelope
Compute power	4–20 W	Bracketed by NASA SmallSat avionics examples
Effective RF return rate	2.2–220 Mbit s ⁻¹	NASA SmallSat ground-system examples
Transmit power	6 W	Fixed simulator assumption
Burst multiplier	1.45×	Contiguous prespecified stressor

An automated audit generated 20,000 tasks before training. All values were finite and positive, all data-flow inequalities held, and the empirical raw-data–workload correlation was 0.660. Figure 3 shows the resulting dependence structure. Constant-rate columns have zero variance and are therefore excluded from correlation coefficients.

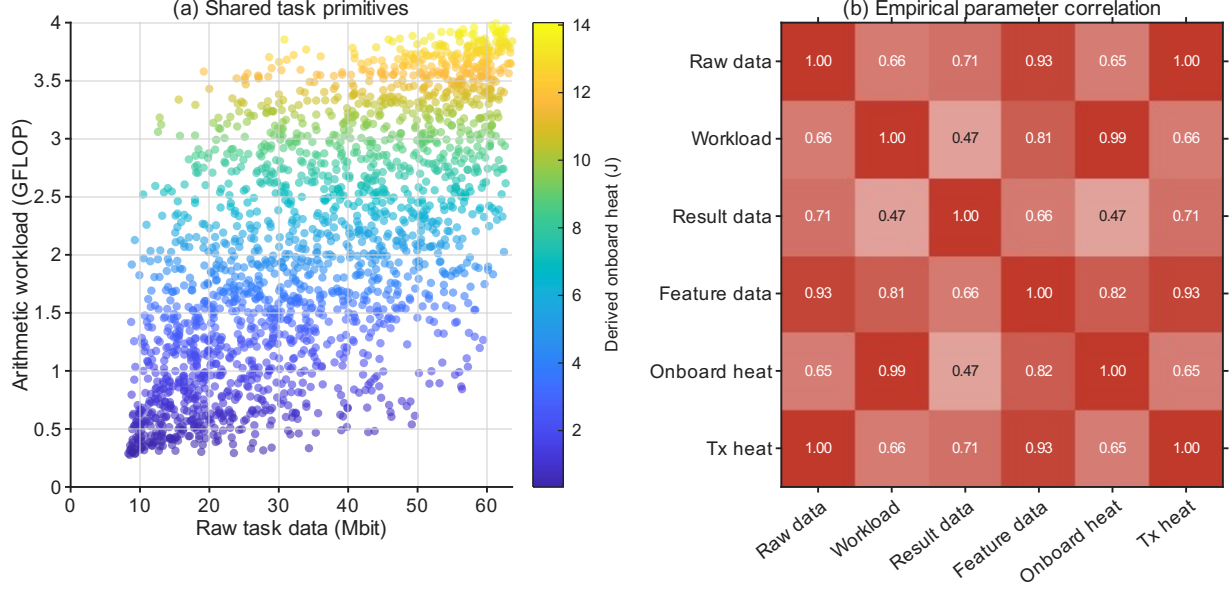


Figure 3: Physical-consistency audit of the revised generator. (a) A legibility sample of 2,000 of 20,000 tasks; color encodes heat derived from compute energy. (b) Correlations computed from all 20,000 tasks. No fitted flight distribution is implied.

3.3 Complete stochastic generator and action-load specification

For independent reconstruction, this subsection states the remaining simulator functions that enter the transition. Let $u_D, \epsilon_W \sim \mathcal{N}(0, 1)$ independently, $u_W = 0.68u_D + \sqrt{1 - 0.68^2}\epsilon_W$, and $g(u) = [1 + \exp(-1.55u)]^{-1}$. Nominal raw volume and workload are

$$D = 8 + 56g(u_D) \text{ Mbit}, \quad W = 0.25 + 3.75g(u_W) \text{ GFLOP}. \quad (7)$$

The burst interval occupies indices $\lfloor T/3 \rfloor$ through $\lfloor T/3 \rfloor + \max(4, \lfloor T/4 \rfloor) - 1$ and multiplies both quantities by 1.45. With independent $U_r, U_f \sim \mathcal{U}(0, 1)$,

$$\begin{aligned} r &= 0.01 + 0.04U_r, & f &= 0.08 + 0.22 \text{clip}[0.65g(u_W) + 0.35U_f, 0, 1], \\ p &= 0.20 + 0.25g(u_W), & D^{res} &= rD, \quad D^{feat} = fD, \quad W^{pre} = pW. \end{aligned} \quad (8)$$

Onboard and ground rates are 4 and 80 GFLOP s^{-1} , respectively. Compute power is $P_c = 4 + 16 \text{clip}(W/4.8, 0, 1)^{0.70}$ W, and transmit power is 6 W. Compute and transmit heat are 0.78 and 0.55 times their corresponding energy use. These definitions generate all 15 coordinates in Table 1.

At reset, a single phase $\phi \sim \mathcal{U}(0, 2\pi)$ generates the exogenous link trace

$$\begin{aligned} \bar{b}_t &= \text{clip} \left\{ \kappa_b [0.62 + 0.23 \sin(2\pi t/24 + \phi) + \epsilon_t^b] - I_{link}, 0.12, 1 \right\}, \\ \bar{\tau}_t &= \text{clip} \left\{ 0.55 + 0.35 \sin(2\pi t/31 + \phi/2) + \epsilon_t^\tau, 0.08, 1 \right\}, \end{aligned} \quad (9)$$

where $\epsilon_t^b \sim \mathcal{N}(0, 0.06^2)$, $\epsilon_t^\tau \sim \mathcal{N}(0, 0.04^2)$, κ_b is the sensitivity multiplier, and $I_{link} = 0.22$ only in the link-limited regime. All episode tasks and link states are drawn at reset from one seeded NumPy generator.

The normalized action loads used in Eq. (12) are stated here in action order $(\alpha_s, \alpha_g, \alpha_{s\&g})$:

$$\begin{aligned}
\hat{w} &= (W, 0.05W, W)/4.8, & \hat{Q} &= (x_1, 0.20x_7, x_{10} + x_{11})/100, \\
\hat{B} &= (x_6, x_9, x_{15})/76.8, \\
\Delta e &= \kappa_e \left(0.012 + 0.042\hat{w}_1, 0.007 + 0.025\hat{B}_2/b, 0.010 + 0.025\hat{w}_3 + 0.018\hat{B}_3/b \right), \\
\Delta q &= \left(0.055\hat{w}_1(1+u), 0.048\hat{B}_2/b, 0.030(\hat{w}_3 + \hat{B}_3/b) \right), \\
\mu &= (0.030(1-u), 0.035b\tau, 0.032(1-0.5u)), \\
\ell &= (0.02, 0.18\hat{B}_2, 0.12\hat{B}_3).
\end{aligned} \tag{10}$$

Here divisions by b use $\max(b, 0.1)$. The utilization targets in Eq. (12) are $(\hat{w}_1, 0.10\hat{B}_2, 0.55\hat{w}_3)$. Initial resources are $(h, e, q, u) = (0.22, 0.88, 0.12, 0.18)$ except that energy-limited episodes use $e = 0.62$ and thermal-stress episodes use $h = 0.42$. Each episode terminates after exactly 64 executed actions; the terminal Bellman multiplier is zero. Energy and other resources remain soft-constrained clipped states, so reaching $e = 0$ does not mask actions or terminate an episode.

4 Sequential Scheduling Formulation

4.1 State and transition

The normalized 21-dimensional state is

$$s_t = [x_t, z_t], \quad z_t = [h_t, e_t, q_t, u_t, b_t, \tau_t], \tag{11}$$

where $x_t \in \mathbb{R}^{15}$ contains the task inputs defined in Table 1. The six resource variables denote normalized heat, remaining energy, queue occupancy, compute utilization, bandwidth, and contact duration.

Executing action a_t changes resources before the next exogenous task arrives. The implemented transition is

$$\begin{aligned}
h_{t+1} &= \text{clip}(0.86h_t + 0.17c\hat{Q}_t^{(a_t)}, 0, 1.3), \\
e_{t+1} &= \text{clip}(e_t + 0.006 - c\Delta e_t^{(a_t)}, 0, 1), \\
q_{t+1} &= \text{clip}(q_t + c(\Delta q_t^{(a_t)} - \mu_t^{(a_t)}), 0, 1.3), \\
u_{t+1} &= \text{clip}(0.68u_t + 0.32c\hat{w}_t^{(a_t)}, 0, 1.2), \\
b_{t+1} &= \text{clip}(\bar{b}_{t+1} - c\ell_t^{(a_t)}, 0.08, 1), \\
\tau_{t+1} &= \text{clip}(\bar{\tau}_{t+1} - 0.10c\hat{B}_t^{(a_t)}, 0.05, 1).
\end{aligned} \tag{12}$$

Here $\hat{Q}$, $\hat{w}$, and $\hat{B}$ are normalized action-specific heat, work, and data demands. The terms Δe , Δq , μ , and ℓ are the deterministic resource-use, queue, service, and link-occupation functions defined in Eq. (10). Exogenous link variables $\bar{b}$ and $\bar{\tau}$ follow Eq. (9) and are sampled before each episode. Coupling $c = 0$ removes action-dependent carryover, $c = 0.5$ weakens it, and $c = 1$ defines full coupling.

4.2 Reward and standard DQN backbone

The reward is the negative immediate cost plus post-decision feasibility penalties:

$$r_t = -c_t^{imm} - c[12(h_{t+1} - 0.78)_+ + 15(0.18 - e_{t+1})_+ + 8(q_{t+1} - 0.75)_+ + 10\mathbf{1}_{a_t \neq \alpha_s}(0.14 - \tau_t)_+]. \quad (13)$$

The DQN uses the unmodified factual target

$$y_t = r_t + \gamma(1 - d_t) \max_{a'} Q_{\bar{\theta}}(s_{t+1}, a'), \quad (14)$$

and factual loss

$$L_{DQN} = \mathcal{H}(Q_{\theta}(s_t, a_t), y_t), \quad (15)$$

where $\mathcal{H}$ denotes the Huber loss. The target network both selects and evaluates the maximizing action, as required by standard DQN.

5 Model-Driven Counterfactual Bellman Advantage Distillation

The proposed method preserves the factual DQN target and factual loss term and adds one training-only auxiliary path. This path supplies targets for unexecuted actions and removes their shared within-state offset. The experiment evaluates this complete auxiliary objective rather than attributing performance to either operation in isolation.

5.1 Side-effect-free full-action preview

For every replayed factual state, the analytical transition evaluates each $a \in \mathcal{A}$:

$$(\tilde{\ell}_{t,a}, \tilde{z}'_{t,a}) = F_{preview}(x_t, z_t, a). \quad (16)$$

Here $\tilde{\ell}_{t,a}$ contains the immediate cost and post-decision feasibility penalty for action a . The preview does not advance time, mutate state, alter trajectories, or consume random numbers. Before action selection it computes only the current-task cost and six post-action resources. The factual action is then executed once, after which the next exogenous task x_{t+1} is revealed and the replay tuple is completed. During a later replay update, all alternatives reuse this observed next task and exogenous link trace. The counterfactual next state therefore replaces only the six endogenous resource coordinates:

$$\tilde{s}'_{t,a} = [x_{t+1}, \tilde{z}'_{t,a}]. \quad (17)$$

Evaluation and deployment never call the preview interface.

5.2 All-action counterfactual Bellman targets

The simulator-exact one-step outcome yields

$$\tilde{y}_{t,a} = -\tilde{\ell}_{t,a} + \gamma(1 - d_t) \max_{a'} Q_{\bar{\theta}}(\tilde{s}'_{t,a}, a'). \quad (18)$$

The targets create no multi-step rollout and insert no counterfactual transition into replay. They are used only after state-wise centering in the complete method below.

5.3 Centered advantage distillation

Absolute targets in Eq. (18) share a bootstrapped state-value offset. Deployment selects $\arg \max_a Q(s, a)$, so a common additive offset cannot change the action. We define predicted and target centered advantages as

$$\begin{aligned}\hat{A}_{t,a} &= Q_{\theta}(s_t, a) - \frac{1}{|\mathcal{A}|} \sum_j Q_{\theta}(s_t, j), \\ \tilde{A}_{t,a} &= \tilde{y}_{t,a} - \frac{1}{|\mathcal{A}|} \sum_j \tilde{y}_{t,j}.\end{aligned}\tag{19}$$

The advantage and total losses are

$$\begin{aligned}L_{adv} &= \frac{1}{|\mathcal{A}|} \sum_{a \in \mathcal{A}} \mathcal{H}(\hat{A}_{t,a}, \tilde{A}_{t,a}), \\ L_{CBAD} &= L_{DQN} + \lambda L_{adv}.\end{aligned}\tag{20}$$

The factual term anchors the absolute Q scale, while L_{adv} teaches all action gaps. The only added algorithmic hyperparameter is λ .

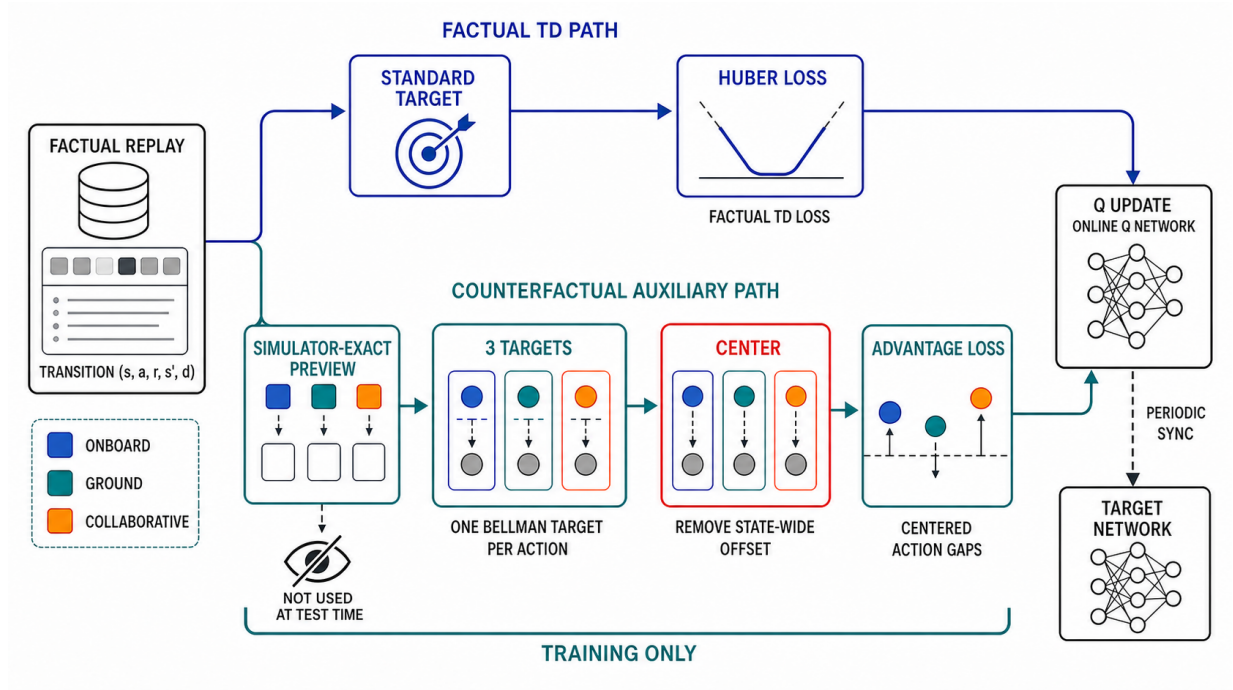


Figure 4: Training-only CBAD mechanism. The upper path retains the factual standard-DQN target and Huber loss. The lower path previews all three actions, constructs three Bellman targets, centers them within state, and distills their relative advantages. Both losses update the online Q-network; the target network is synchronized only at the shared fixed interval. Preview outputs are unavailable during evaluation and deployment.

Algorithm 1: Pure-DQN training with CBAD

1. Initialize uniform replay $\mathcal{D}$, online Q_θ , and target $Q_{\bar{\theta}} \leftarrow Q_\theta$.
2. At each real step, observe s_t and preview current-task outcomes $(\tilde{\ell}_{t,a}, \tilde{z}'_{t,a})$ for all actions without side effects.
3. Select a_t by the shared epsilon-greedy schedule and execute it once. After x_{t+1} is revealed, store the factual tuple plus the previously computed previews.
4. When an update is due, sample one uniform mini-batch, compute the factual loss, compute Eqs. (18) and (19), and update θ using Eq. (20).
5. Synchronize $Q_{\bar{\theta}}$ at the shared fixed interval.

Training previews are never used for evaluation or deployment.

6 Experimental Design and Settings

6.1 Design principles

The design separates four questions. Standard DQN versus CBAD tests the complete centered counterfactual objective against an unchanged factual DQN backbone. An uncentered full-action Q auxiliary target and a centered immediate-cost auxiliary target distinguish state-wise centering and Bellman bootstrapping from the effect of supplying three labels per state. Double DQN and Double-CBAD test whether the auxiliary objective transfers when action selection and target evaluation are decoupled [25]. Coupling controls and planners with $H \in \{1, 2, 4, 6\}$ test whether long-horizon learning remains useful as delayed resource effects and planning depth change.

Standard DQN and CBAD share every component except the centered counterfactual auxiliary objective. They use the same input, network, replay, exploration, optimizer, update cadence, target synchronization, and gradient clipping. The full-action Q control applies the same auxiliary weight directly to the three uncentered Bellman targets. The immediate-advantage control centers the negative analytical one-step costs but omits their bootstrapped future values. Double DQN and Double-CBAD use online-network action selection with target-network evaluation in both factual and auxiliary Bellman targets. Dueling networks, prioritized replay, multi-step returns, noisy networks, and distributional value learning remain outside the comparison.

The auxiliary weight was fixed before the physical-consistency experiment. A preceding calibration with model seeds 700–703 compared $\lambda \in \{0.1, 0.3, 1, 3\}$. It selected $\lambda = 0.3$ by maximizing the worst relative CBAD improvement across five fully coupled calibration regimes, with the smaller weight as the tie-break. Because that calibration used the preceding simulator version, it is a prior specification rather than evidence from the physical-consistency generator. The present confirmation and all new controls use seeds 800–819 without post hoc removal, replacement, or selection. Every learned policy was trained only on the nominal $c = 1$ distribution.

6.2 Training configuration

Each network has 21 inputs, hidden widths 128–128–64, ReLU activations, and three outputs. Every model receives 600 episodes of 64 tasks, totaling 38,400 real interactions. Fixed settings are $\gamma = 0.97$, learning rate 3×10^{-4} , batch size 128, replay capacity 100,000, and gradient-norm limit 10. Training begins after 128 transitions and performs one update every four interactions.

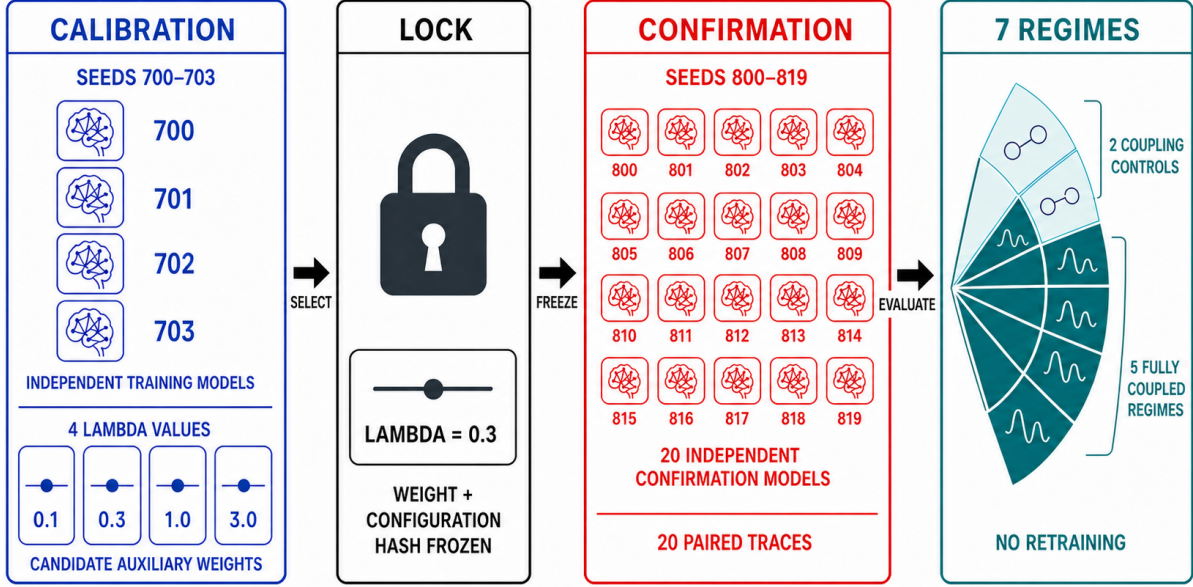


Figure 5: Prespecification and confirmation protocol. A preceding-simulator calibration with four seeds selected $\lambda = 0.3$ before the physical-consistency experiment. Twenty disjoint confirmation seeds, 800–819, are each evaluated on 20 shared held-out traces in seven regimes, with no physical-generator tuning or seed exclusion.

The checkpoint field `target_interval=250` is evaluated only on update calls; because updates occur at interaction counts divisible by four, the two divisibility conditions coincide every 500 real interactions. Epsilon decreases linearly from 1 to 0.05 over 13,440 interactions.

The contextual bandit uses the same network, optimizer, exploration schedule, and interaction budget, with $\gamma = 0$. It therefore learns the modeled one-step reward without a long-horizon Bellman term. Uniform replay stores only one-step factual transitions, with preview outputs attached as auxiliary training data.

All reported reviewer-revision experiments use the physically coupled generator described above. Link bandwidth and contact follow bounded noisy periodic traces generated at reset. Burst and thermal-stress episodes multiply raw data and workload by 1.45 during one contiguous interval; all dependent heat, energy, latency, and transmission quantities are recomputed from those primitives. The original independently sampled generator is retained only as a historical reproducibility mode and is not used for the revised results.

6.3 Seven evaluation regimes

The seven regimes separate coupling controls from fully coupled operating conditions:

1. **Static** ($c = 0$): removes action-dependent state transitions and delayed penalties.
2. **Reduced coupling** ($c = 0.5$): halves action-dependent resource effects.
3. **Nominal** ($c = 1$): matches the fully coupled training distribution.

4. **Burst:** increases contiguous compute and thermal demand.
5. **Link-limited:** reduces the available bandwidth trace by 0.22 before clipping.
6. **Energy-limited:** lowers initial energy from 0.88 to 0.62.
7. **Thermal-stress:** raises initial heat from 0.22 to 0.42 and includes the burst interval.

The first two regimes control temporal coupling. The remaining five retain full coupling. Models trained at $c = 1$ are transferred to both controls without retraining. Their control performance therefore measures a boundary and transfer mismatch, rather than the best policy attainable at lower coupling.

6.4 Planning comparator, outcomes, and inference

The deployment comparison includes immediate-cost argmin, a contextual bandit, a resource-threshold heuristic, standard DQN, MD-CBAD-DQN, and exhaustive finite-horizon model-based planning. At every real decision, a planner enumerates all 3^H action sequences over the next H known tasks and link states, minimizes discounted modeled cost with $\gamma = 0.97$, executes only the first action, and replans. The primary table retains $H = 4$. A separate nominal analysis evaluates $H \in \{1, 2, 4, 6\}$ on 400 unique held-out traces and reports wall-clock cost per decision. Each planner is exact only for its stated perfect-forecast objective, not for the 64-step global problem. Fixed onboard and fixed ground policies serve as supplemental controls.

The primary outcome is episode total cost. Secondary outcomes are immediate cost, delayed penalty, energy use, modeled latency, transmitted data, threshold-exposure steps, action fractions, planner wall-clock time, maximum heat, and minimum remaining energy. Every learned comparison uses a prespecified common-random-number block. The same model-seed number initializes network weights and controls episode task seeds, epsilon-exploration draws, and replay-index sampling in both algorithms. Trajectories may diverge after different greedy actions, but the matched seed remains the randomization block. Each seed is evaluated on the same 20 held-out traces and traces are averaged within seed, giving $n = 20$ blocks per method. Two-sided 95% percentile intervals use 10,000 paired bootstrap resamples with seed 20260829. Exact two-sided paired sign tests evaluate directional consistency, with Holm adjustment across the five fully coupled primary comparisons. We also report every seed-level difference and the leave-one-seed-out range. As a conservative sensitivity check, an independent-seed bootstrap resamples the two sets of 20 seed means separately and ignores the matching. Test traces and bootstrap draws are never treated as independent model replicates.

6.5 Prespecified sensitivity and training diagnostics

Without retraining or selecting models, the nominal evaluation was repeated under eight profiles: the reference configuration; compute demand $\pm 20\%$; raw data volume $\pm 20\%$; energy use $+20\%$; heat load $+20\%$; and link capacity -20% . Each profile retained the same 20 models and the same held-out trace namespace as the primary experiment, with Holm adjustment across the eight profiles. Training logs record every episode for all 20 seeds. Curves report a causal 25-episode moving mean and seed-level mean $\pm$ SD for total cost, energy use, latency, and delayed penalty. These diagnostics describe optimization trajectories; confirmation claims remain based on held-out traces.

6.6 Preview-model misspecification test

The main experiment assumes that the analytical preview matches the simulator transition. To test this dependence, two additional CBAD model sets were trained without changing the factual environment or deployment evaluation. One set scaled preview one-step costs (immediate cost plus modeled penalty) and action-induced resource changes by 0.85, while the other used 1.15. Resource predictions were clipped to the same physical bounds after scaling around the current resource state. Each condition used the same 20 seeds, training budget, network, replay, and held-out trace namespace as the main experiment. These scales were fixed as symmetric stress tests and were not used for model selection.

6.7 Reproducible result entry point

The core result entry point is `python -m md_cbad_dqn.viewer_experiments all --seeds 800-819 --resume`. The second-round controls use `python -m md_cbad_dqn.extended_revision --seeds 800-819 --stage all --resume`. The first command trains or loads 60 primary learned-policy checkpoints plus 40 preview-error checkpoints. The second reuses standard DQN and CBAD and trains 80 component and Double-DQN checkpoints. It then evaluates six learned objectives, performs the independent-seed bootstrap, and evaluates four planning depths on 400 unique traces. The command `python -m md_cbad_dqn.transition_audit` writes a fixed-seed, all-action example in which preview and executed costs and six resources agree coordinate by coordinate. Unit tests repeat this equality at both resource clipping boundaries and on the terminal step. Experiments used Python 3.13.9, NumPy 2.4.1, pandas 3.0.3, PyTorch 2.8.0 with CUDA 12.9, an NVIDIA GeForce RTX 5070 Ti, and Windows 11. Planning wall time was measured on a 12th Gen Intel Core i9-12900K with 24 logical processors after one untimed warm-up episode. Values report the mean over all timed decisions. Machine-readable metadata record the resolved variants, seeds, budgets, randomization blocks, result files, and runtime environment. Table scripts consume these files directly.

7 Results

7.1 Physical consistency preserves the CBAD-DQN effect

Table 3 and Fig. 6 summarize the revised primary outcome. Values are mean $\pm$ SD across 20 independent model seeds after averaging 20 shared traces within each seed.

Table 3: Episode total cost for deployment comparators under the physically coupled generator. All learned policies were trained only at nominal $c = 1$; static and reduced-coupling rows are transfers without retraining. MPC-4 is exact only for its stated four-step perfect-forecast objective. Lower is better; bold marks the lowest mean.

Regime	Argmin	MPC-4	Bandit	Threshold	DQN	CBAD
Static ($c = 0$)	24.55 $\pm$ 0.21	24.55 $\pm$ 0.21	24.69 $\pm$ 0.20	24.74 $\pm$ 0.25	27.46 $\pm$ 1.34	27.43 $\pm$ 1.00
Reduced ($c = 0.5$)	26.65 $\pm$ 0.63	25.70 $\pm$ 0.26	26.04 $\pm$ 0.37	26.43 $\pm$ 0.38	28.01 $\pm$ 0.72	27.80 $\pm$ 0.55
Nominal	193.42 $\pm$ 8.33	90.83 $\pm$ 1.71	100.63 $\pm$ 3.11	188.08 $\pm$ 6.41	78.80 $\pm$ 2.46	77.45 $\pm$ 1.86
Burst	241.20 $\pm$ 6.84	108.35 $\pm$ 1.99	129.31 $\pm$ 8.08	238.74 $\pm$ 6.56	97.99 $\pm$ 2.45	96.95 $\pm$ 2.12
Link-limited	272.02 $\pm$ 7.98	115.73 $\pm$ 1.92	137.55 $\pm$ 5.87	227.25 $\pm$ 8.36	102.86 $\pm$ 1.93	101.96 $\pm$ 1.97
Energy-limited	225.31 $\pm$ 7.73	126.00 $\pm$ 1.58	134.57 $\pm$ 2.93	246.46 $\pm$ 6.45	119.60 $\pm$ 2.42	118.15 $\pm$ 1.68
Thermal-stress	247.58 $\pm$ 6.25	109.20 $\pm$ 2.01	130.34 $\pm$ 7.81	241.80 $\pm$ 6.26	98.19 $\pm$ 2.46	97.20 $\pm$ 2.16

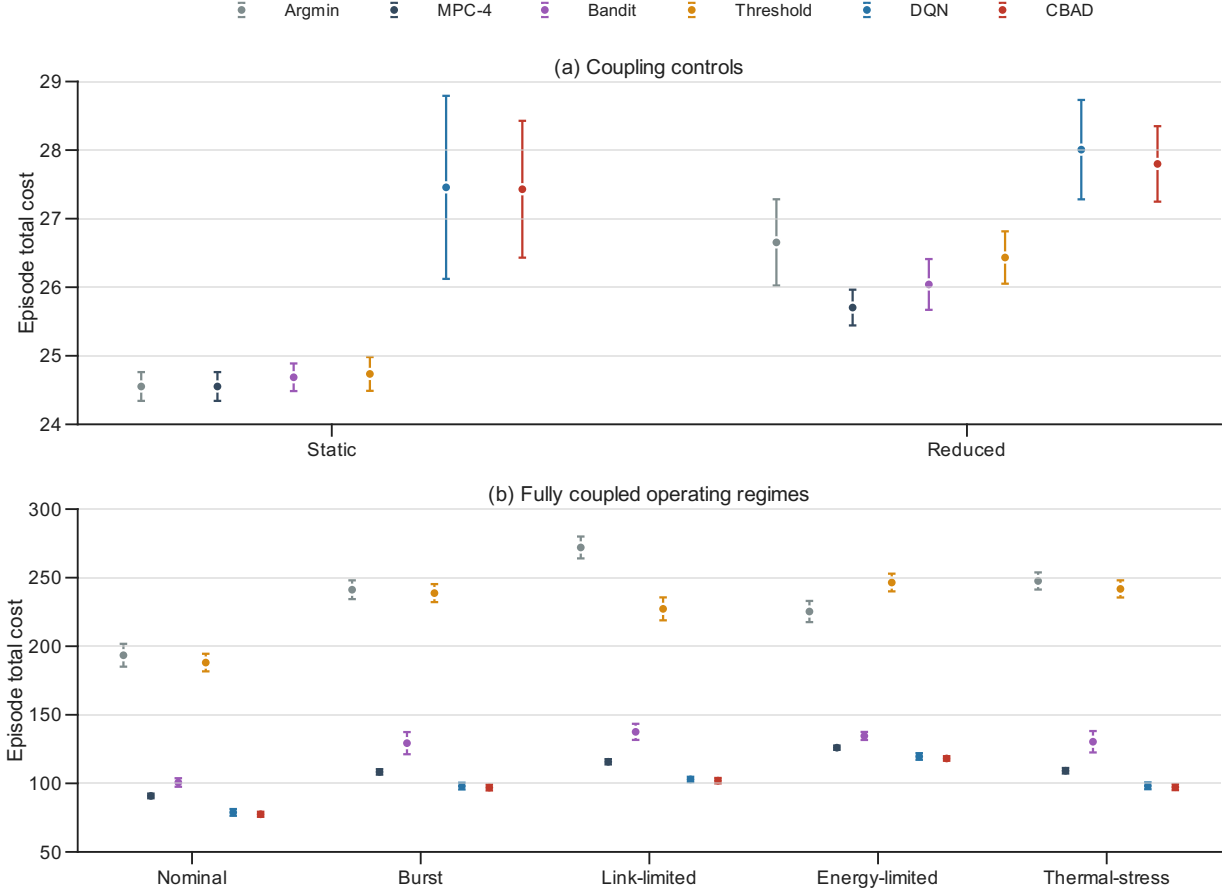


Figure 6: Seven-regime performance under physically coupled task generation. Error bars are SD across 20 model-seed blocks after within-seed trace averaging. All learned models were trained at nominal $c = 1$; control rows are transfers without retraining. The ordinate changes between panels and panel heights should not be compared directly.

Table 4: Seed-level primary inference for CBAD minus standard DQN in the five fully coupled regimes. Negative differences favor CBAD. The Holm family contains these five prespecified comparisons. LOO denotes the range of leave-one-seed-out mean differences.

Regime	Mean difference	95% interval	Negative seeds	Holm p	LOO range
Nominal	-1.36	[-2.01, -0.81]	16/20	0.03545	[-1.44, -1.14]
Burst	-1.04	[-1.58, -0.56]	17/20	0.01031	[-1.12, -0.86]
Link-limited	-0.90	[-1.30, -0.53]	16/20	0.03545	[-0.97, -0.79]
Energy-limited	-1.45	[-2.18, -0.85]	18/20	0.002012	[-1.55, -1.20]
Thermal-stress	-0.99	[-1.56, -0.50]	16/20	0.03545	[-1.08, -0.81]

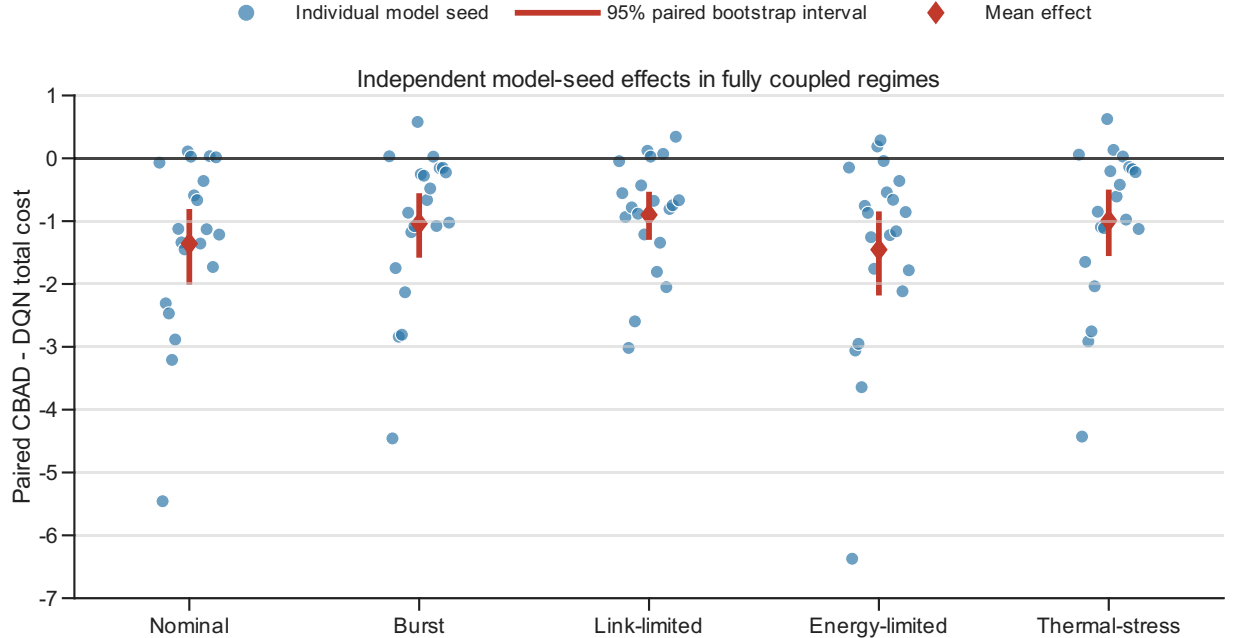


Figure 7: Paired total-cost differences for all 20 independent model seeds. Blue points show individual seed-level effects after averaging 20 shared traces. Red diamonds and vertical lines show mean differences and paired 95% bootstrap intervals.

Table 5: Sensitivity of the primary CBAD–DQN comparison to ignoring the prespecified model-seed matching. The two groups of 20 seed means are resampled independently. These intervals quantify the loss of precision when the common-random-number blocks are discarded and are not the primary analysis.

Regime	Mean difference	Unpaired 95% interval	Excludes zero
Nominal	-1.36	[-2.67, -0.06]	Yes
Burst	-1.04	[-2.41, 0.36]	No
Link-limited	-0.90	[-2.08, 0.31]	No
Energy-limited	-1.45	[-2.71, -0.21]	Yes
Thermal-stress	-0.99	[-2.41, 0.42]	No

Across the five fully coupled regimes, MD-CBAD-DQN had the lowest mean among all evaluated deployment methods and reduced standard-DQN cost by 0.87%–1.72%; every paired 95% interval lay below zero. CBAD was lower for 16–18 of the 20 seed blocks in each regime, the largest Holm-adjusted sign-test value was 0.03545, and every leave-one-seed-out mean remained negative (Table 4). Nominal mean cost decreased from 78.80 for standard DQN to 77.45 for CBAD. The independent-seed bootstrap was wider and excluded zero only in a subset of regimes (Table 5). The direction therefore remains consistent, but the all-regime inferential statement depends on the prespecified common-random-number design. This replication changes both the task generator and model training, so it is a new physical-consistency experiment rather than a re-labeling of the earlier independent-range simulation.

At $c = 0$, immediate argmin and MPC-4 were identical because actions had no delayed effect. At $c = 0.5$, MPC-4 had the lowest mean. In contrast, CBAD outperformed MPC-4 in all five

fully coupled regimes. Nominal MPC-4 cost was 90.83, despite perfect four-task forecasts, and planning required 4.81 ms per decision on the evaluation workstation. This comparison alone does not identify the cause of the gap, so the planning-depth analysis below separates horizon and computation.

7.2 Component controls identify Bellman bootstrapping, but not centering, as the supported distinction

Table 6: Nominal component and backbone controls. All models use 20 prespecified model-seed blocks, identical real-interaction budgets, and the same held-out traces. Full-action Q applies uncentered Bellman targets; immediate advantage centers analytical immediate costs without bootstrapping. Paired differences are relative to the matched reference named in the third column.

Training objective	Mean cost $\pm$ SD	Matched reference	Paired difference [95% interval]
Standard DQN	78.80 $\pm$ 2.46	–	–
Full-action Q auxiliary	77.51 $\pm$ 1.74	Standard DQN	-1.29 [-1.98, -0.73]
Immediate advantage auxiliary	78.03 $\pm$ 1.75	Standard DQN	-0.77 [-1.44, -0.18]
CBAD	77.45 $\pm$ 1.86	Standard DQN	-1.36 [-1.98, -0.81]
Double DQN	78.50 $\pm$ 2.51	–	–
Double-CBAD	77.46 $\pm$ 1.78	Double DQN	-1.04 [-1.90, -0.38]

Table 6 separates three explanations that the standard-DQN comparison cannot resolve. Uncentered full-action Q supervision reduced nominal cost by 1.29 relative to standard DQN, compared with 1.36 for CBAD; the direct CBAD-minus-full-action-Q difference was -0.07 with a paired 95% interval of $[-0.32, 0.20]$. Centering therefore showed no detectable increment under this design. The immediate-advantage control improved by 0.77 relative to standard DQN, but CBAD was lower by 0.58, with an interval of $[-1.00, -0.14]$. This pattern attributes the stronger effect to action-specific bootstrapped values rather than to three immediate labels alone. Double-CBAD was lower than Double DQN by 1.04, with an interval of $[-1.90, -0.38]$; however, the Holm-adjusted sign-test value was 0.059, so this transfer result is directional rather than confirmatory. These are fixed objective controls, not a post hoc search over alternative algorithms.

7.3 Planning depth improves cost at a rapidly increasing computational price

Table 7: Nominal receding-horizon planning over 400 unique held-out traces. Each planner has perfect knowledge of the next H tasks and exogenous link states and exactly enumerates 3^H sequences before every action. Runtime was measured on the same workstation and includes only action selection after environment construction.

Planning horizon	Enumerated sequences	Mean cost	Time per decision (ms)	Held-out traces
1	3	99.33	0.13	400
2	9	92.80	0.49	400
4	81	90.83	4.70	400
6	729	89.35	41.60	400

Increasing planning depth reduced nominal mean cost from 99.33 at $H = 1$ to 92.80, 90.83, and 89.35 at $H = 2, 4$, and 6, respectively. The corresponding mean decision time increased from

0.13 ms to 0.49, 4.70, and 41.60 ms. The additional 1.48 cost-unit reduction from $H = 4$ to $H = 6$ therefore required an almost ninefold increase in action-selection time. Table 7 should not be interpreted as a comparison with optimized approximate planning, dynamic programming, or mixed-integer methods. It quantifies only the exact-enumeration frontier implemented here, and the learned-policy comparison remains bounded to these transparent objectives.

Table 8: Engineering outcome decomposition for CBAD relative to standard DQN. Delta columns report CBAD minus DQN. Negative values denote lower use or cost. Low-energy steps count decisions with remaining energy below 0.18; they are soft-threshold exposures, not certified safety violations.

Regime	Δ immediate	Δ penalty	Δ energy	Δ latency (s)	Δ data (Mbit)
Nominal	-0.51	-0.84	-0.010	-0.16	58.4
Link-limited	-0.46	-0.44	-0.008	-0.18	23.2
Energy-limited	-0.76	-0.69	-0.013	-0.24	80.8
Thermal-stress	-0.58	-0.41	-0.009	-0.18	73.6

Regime	Low-energy steps DQN/CBAD	Min. energy DQN/CBAD	Max. heat DQN/CBAD
Nominal	23.7/23.4	0.00/0.00	0.192/0.192
Link-limited	30.2/30.1	0.00/0.00	0.193/0.193
Energy-limited	38.8/38.5	0.00/0.00	0.193/0.192
Thermal-stress	28.8/28.7	0.00/0.00	0.364/0.364

Table 8 separates the composite objective into its modeled consequences. Across the displayed regimes, CBAD reduced both immediate cost and delayed penalty, used slightly less modeled energy, and reduced modeled latency, while transmitting more data. The result is therefore a trade-off rather than a uniform reduction in every resource. The low-energy count is reported as threshold exposure because the simulator applies a soft penalty rather than a hard feasibility constraint. Both learned policies reached zero remaining energy and no evaluated episode was threshold-free, so these experiments do not establish hard-constrained deployability. Thermal-stress increases the initial heat and contiguous load, but the tested policies did not cross the heat threshold. It therefore probes elevated thermal load rather than certifying operation at a thermal safety boundary.

7.4 Training diagnostics show stabilization of cost, energy, and latency

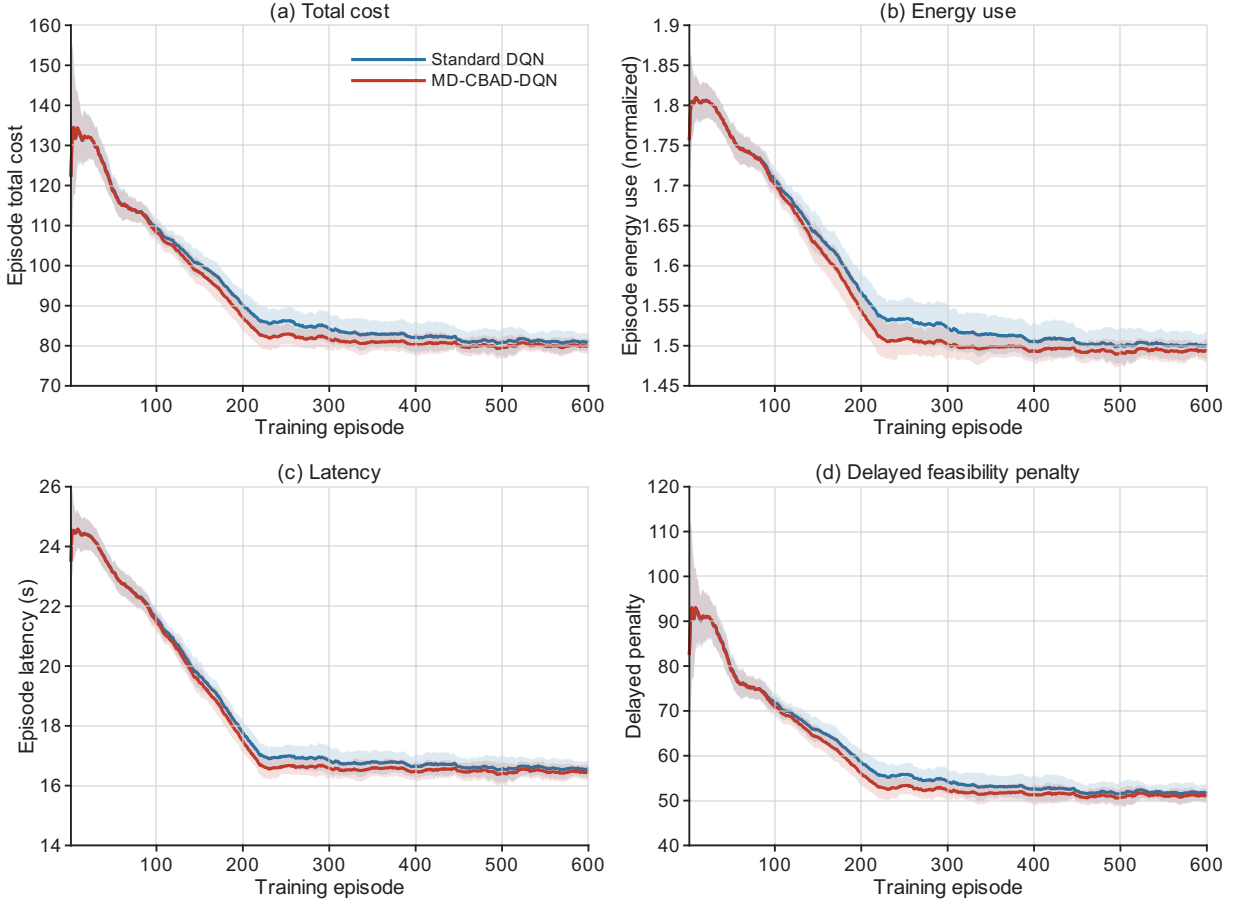


Figure 8: Training diagnostics for standard DQN and MD-CBAD-DQN. Lines are means across 20 model seeds after a causal 25-episode moving average; shaded regions are ± 1 SD. Curves describe the training distribution and are not used as independent test observations.

All four logged quantities declined rapidly during the first approximately 220 episodes and then fluctuated around a stable range (Fig. 8). Across seeds and episodes 501–600, CBAD averaged 80.09 total cost, 1.49 normalized energy use, 16.48 s latency, and 51.18 delayed penalty, compared with 81.12, 1.50, 16.59 s, and 51.85 for standard DQN. These curves answer the optimization question without reintroducing the undefined “accuracy” metric. They show empirical stabilization, not mathematical convergence.

7.5 The result is robust within the tested parameter envelope

Table 9: Prespecified parameter sensitivity for CBAD minus standard DQN. Negative differences favor CBAD; $n = 20$ independent paired model seeds in each profile. Holm adjustment covers the eight displayed profiles.

Parameter profile	Mean difference	95% interval	Improvement	Holm p
Reference	-1.36	[-2.00, -0.81]	1.72%	0.03545
Compute demand -20%	-1.13	[-1.58, -0.68]	1.81%	0.01804
Compute demand $+20\%$	-1.47	[-2.33, -0.74]	1.62%	0.03545
Input data -20%	-2.27	[-3.58, -1.19]	3.61%	0.01804
Input data $+20\%$	-0.80	[-1.21, -0.42]	0.90%	0.04139
Energy use $+20\%$	-1.22	[-1.79, -0.71]	1.16%	0.00322
Heat load $+20\%$	-1.36	[-2.00, -0.81]	1.73%	0.01804
Link capacity -20%	-1.06	[-1.50, -0.64]	1.18%	0.01804

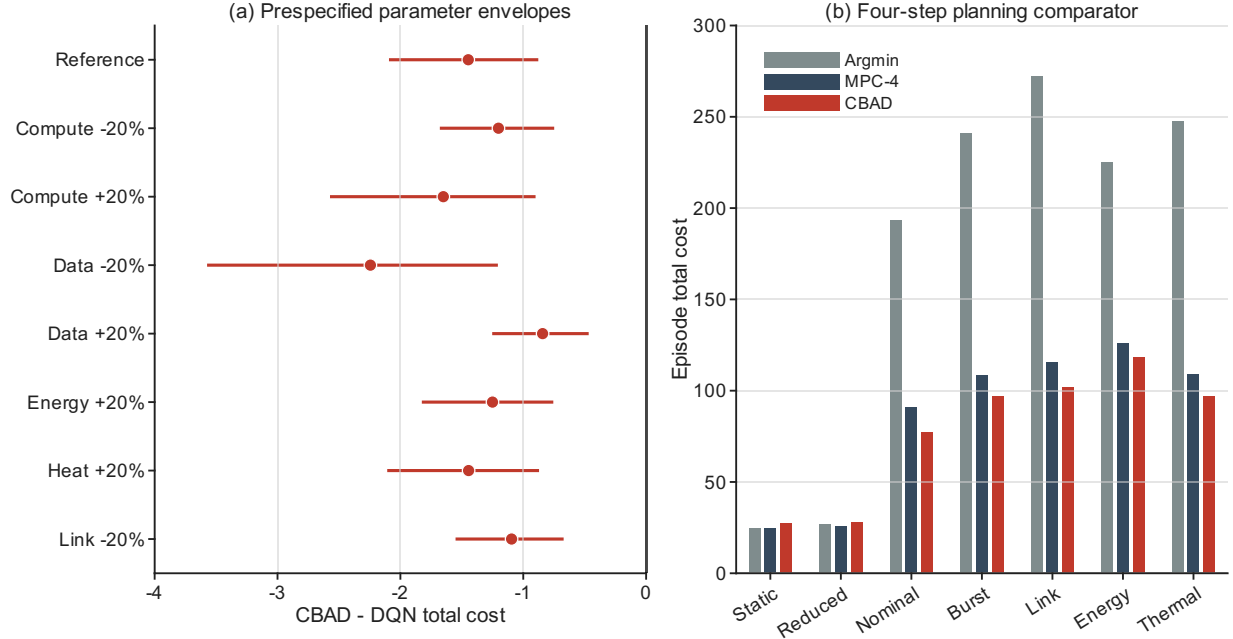


Figure 9: Parameter robustness and planning boundary. (a) Mean paired CBAD-DQN differences with 95% paired bootstrap intervals. (b) Immediate, four-step rolling, and learned scheduling across coupling and stress regimes.

The CBAD-DQN interval remained below zero in all eight profiles, with relative reductions from 0.90% to 3.61%. The largest tested effect occurred when raw data volume decreased by 20%, whereas the smallest occurred when it increased by 20%. This analysis supports robustness only within these prespecified one-factor envelopes; it does not validate arbitrary hardware or mission distributions.

7.6 Moderate preview error preserves a bounded benefit

Table 10: Effect of training-time preview misspecification. Differences are relative to the same standard-DQN models evaluated in the factual simulator. Scaling applies only to CBAD’s training-only preview cost and action-induced resource changes.

Training preview	Regime	Mean difference	95% interval	Negative seeds	Holm p
Simulator-exact preview	Nominal	-1.36	[-2.01, -0.81]	16/20	0.03545
	Burst	-1.04	[-1.58, -0.56]	17/20	0.01031
	Link-limited	-0.90	[-1.30, -0.53]	16/20	0.03545
	Energy-limited	-1.45	[-2.18, -0.85]	18/20	0.002012
	Thermal-stress	-0.99	[-1.54, -0.50]	16/20	0.03545
Consequences scaled to 0.85	Nominal	-1.43	[-2.11, -0.88]	19/20	0.0002003
	Burst	-1.06	[-1.64, -0.59]	19/20	0.0002003
	Link-limited	-1.03	[-1.47, -0.63]	17/20	0.005154
	Energy-limited	-1.49	[-2.28, -0.82]	17/20	0.005154
	Thermal-stress	-1.05	[-1.65, -0.56]	19/20	0.0002003
Consequences scaled to 1.15	Nominal	-1.28	[-2.02, -0.63]	15/20	0.1656
	Burst	-0.91	[-1.54, -0.36]	15/20	0.1656
	Link-limited	-0.86	[-1.40, -0.36]	15/20	0.1656
	Energy-limited	-1.53	[-2.34, -0.82]	18/20	0.002012
	Thermal-stress	-0.81	[-1.48, -0.23]	13/20	0.2632

Table 10 tests whether the auxiliary signal remains useful when its modeled one-step consequences are uniformly under- or over-scaled by 15%. Both misspecified model sets retained negative mean differences and paired intervals below zero in all five regimes; their Holm-adjusted sign-test values were at most 0.2632. The 1.15 scale attenuated the mean benefit in four regimes, whereas the 0.85 scale produced effects close to the simulator-exact preview. This test does not represent arbitrary structural or action-specific error, but it separates simulator-internal exactness from moderate uniform calibration error and shows that the reported advantage survives this fixed perturbation.

7.7 The learned policy adapts its action mix

CBAD did not collapse to one processing mode. It increased onboard execution under link limitation and shifted toward ground or collaborative execution as the inherited state permitted. These action fractions are descriptive consequences of the learned policy, not additional inferential endpoints.

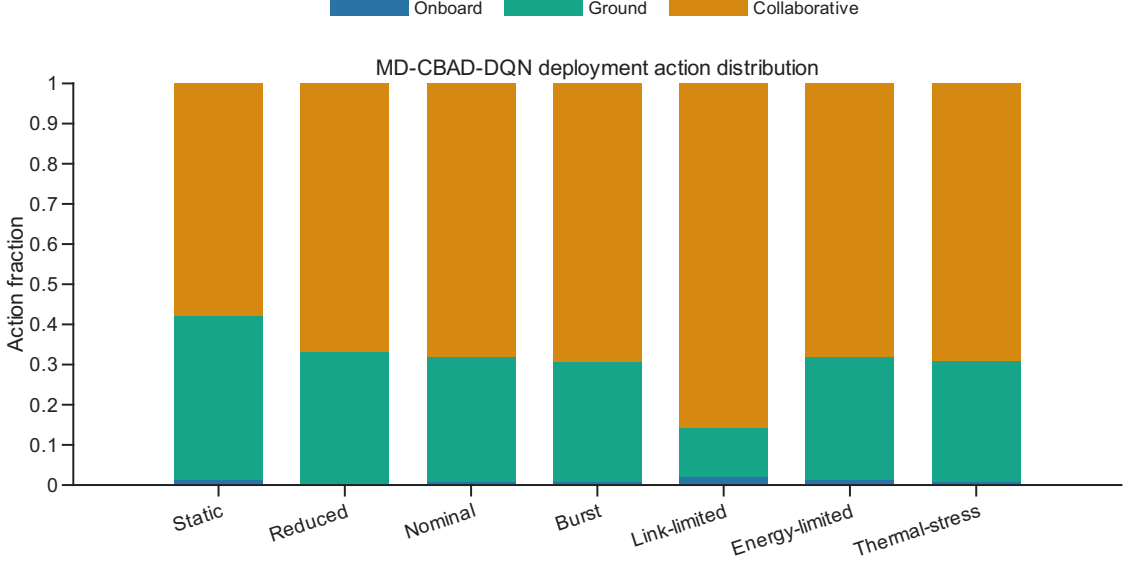


Figure 10: Mean deployment action fractions for MD-CBAD-DQN under the physically coupled generator.

8 Discussion and Limitations

The revised evidence supports full-action Bellman auxiliary supervision against a strictly matched standard-DQN backbone across five fully coupled regimes. The centered CBAD form did not separate from its uncentered full-action Q control, so the present data do not attribute the gain specifically to centering. By contrast, the smaller immediate-advantage effect supports retaining action-specific future values. The Double-DQN pair was directionally consistent but not confirmatory after family adjustment. RL is not required for the static control, where immediate minimization is exact after action-dependent carryover is removed.

The planning-depth comparison bounds the motivation more directly. Increasing H from four to six reduced mean cost by 1.48 but multiplied decision time by 8.9, exposing a diminishing-return frontier within exact enumeration. It still does not establish the best horizon or compare against approximate tree search, mixed-integer programming, or dynamic programming with discretized resources. We therefore claim an advantage only over the reported exact finite-horizon objectives, not over planning or traditional optimization in general.

The matched DQN backbone remains an attribution design rather than a broad reinforcement-learning benchmark. Double DQN is included as one prespecified stronger target construction and receives its own matched Double-CBAD arm. Dueling networks, prioritized replay, multi-step returns, noisy networks, and distributional value learning remain untested. Accordingly, the evidence does not establish superiority over every enhanced value-learning method.

The new generator removes internally contradictory independent samples by deriving heat, energy, latency, and transmitted volume from common primitives. NASA and CCSDS sources bound selected engineering assumptions, while automated checks enforce data-flow inequalities. Nevertheless, the joint distribution is synthetic rather than fitted to mission telemetry. NASA itself cautions that component performance is mission-specific [26]. The sensitivity study covers only prespecified one-factor changes, not simultaneous uncertainty or structural model error.

Matched training components attribute the simulator-level comparisons to the stated objective changes, but external validity remains limited. “Simulator-exact” means one-step equality only with

the specified deterministic program; it does not imply physical fidelity to a spacecraft. The topology remains one satellite and one ground path, the action space contains three modes, the utility weights are not mission-fitted, and inference remains simulation-based. Both principal learned policies reach zero remaining energy in some episodes because feasibility is represented by a soft penalty. Future work should impose action masking or constrained control, use flight or hardware-in-the-loop traces, expand to multiple satellites and contacts, and compare scalable approximate planning.

9 Conclusion

We introduced MD-CBAD-DQN to exploit one-step analytical resource models while retaining the factual DQN loss. In the defined synthetic soft-constrained simulator, CBAD had lower paired mean cost than standard DQN across five fully coupled regimes. Component controls localize the supported contribution to full-action Bellman supervision rather than centering alone; the matched Double-DQN result is directional after multiplicity control. Planning depths from one to six steps expose the cost of stronger exact look-ahead. The evidence establishes a simulator-level training-objective result, not hard-constrained flight readiness or general superiority to long-horizon planning.

Funding

This work was supported by the National Key Research and Development Program of China under Grants 2022YFF0503904 and 2022YFD2401202. Additional support came from the Shenzhen Higher Education Institutions Stabilization Support Program under Grant GXWD 20220811163556003. The National Natural Science Foundation of China supported this work under Grant 62202127, and the Shenzhen Natural Science Foundation under Grant JCYJ 20241202123731040. Further support came from the Guangxi Science and Technology Base and Talent Special Project under Grant Gui Ke AD25069103.

Data and Code Availability

The accompanying versioned repository contains the simulator, primary, preview-error, component-control, and Double-DQN checkpoints, raw evaluations, seed-level summaries, fixed-seed inference, direct Python dependencies, MATLAB figure source, and LaTeX-table generators under `md_cbad_dqn/`. Machine-readable validation and metadata files record expected rows, model seeds, training budgets, randomization blocks, package versions, and result hashes. The commands in the reproducibility subsection regenerate the result chain. **AUTHOR ARCHIVE IDENTIFIER TO BE INSERTED BEFORE SUBMISSION.**

References

- [1] Yanhua Pang, Yamin Zhang, Yi Wang, Xiaofeng Wei, and Bo Chen. Socnet: A lightweight and fine-grained object recognition network for satellite on-orbit computing. *IEEE Transactions on Geoscience and Remote Sensing*, 60:1–13, 2022.
- [2] Yanhua Pang, Yamin Zhang, Qinglei Kong, Yi Wang, Bo Chen, and Xibin Cao. Socdet: A lightweight and accurate oriented object detection network for satellite on-orbit computing. *IEEE Transactions on Geoscience and Remote Sensing*, 61:1–15, 2023.

- [3] Yanhua Pang, Yamin Zhang, Yi Wang, Xiaofeng Wei, and Bo Chen. Exploring model compression limits and laws: A pyramid knowledge distillation framework for satellite-on-orbit object recognition. *IEEE Transactions on Geoscience and Remote Sensing*, 62:1–13, 2024.
- [4] Yanhua Pang, Libo Yao, Yiping Luo, Chengguo Dong, Qinglei Kong, and Bo Chen. Repsvit: An efficient vision transformer based on spiking neural networks for object recognition in satellite on-orbit remote sensing images. *IEEE Transactions on Geoscience and Remote Sensing*, 62:1–16, 2024.
- [5] Yin Zhang, Mu Ye, Guiyi Zhu, Yong Liu, Pengyu Guo, and Junhua Yan. Ffca-yolo for small object detection in remote sensing images. *IEEE Transactions on Geoscience and Remote Sensing*, 62:1–15, 2024.
- [6] Emiliano Casalicchio and Danilo Magliarisi. Decentralized task scheduling in satellite edge computing. In *2024 9th International Conference on Fog and Mobile Edge Computing (FMEC)*, pages 154–161. IEEE, 2024.
- [7] Junhong Kim, Jaemyung Ahn, Han-Lim Choi, and Doo-Hyun Cho. Task scheduling of agile satellites with transition time and stereoscopic imaging constraints. *Journal of Aerospace Information Systems*, 17(6):285–293, 2020.
- [8] Weisen Liu, Zeqi Lai, Qian Wu, Hewu Li, Qi Zhang, Zonglun Li, Yuanjie Li, and Jun Liu. In-orbit processing or not? sunlight-aware task scheduling for energy-efficient space edge computing networks. In *IEEE INFOCOM 2024-IEEE Conference on Computer Communications*, pages 881–890. IEEE, 2024.
- [9] Qiangqiang Jiang, Haipeng Wang, Qinglei Kong, Yamin Zhang, and Bo Chen. On-orbit remote sensing image processing complex task scheduling model based on heterogeneous multiprocessor. *IEEE Transactions on Geoscience and Remote Sensing*, 61:1–18, 2023.
- [10] Qiangqiang Jiang, Lujie Zheng, Yu Zhou, Hao Liu, Qinglei Kong, Yamin Zhang, and Bo Chen. Efficient on-orbit remote sensing imagery processing via satellite edge computing resource scheduling optimization. *IEEE Transactions on Geoscience and Remote Sensing*, 2025.
- [11] Xin Shen, Zezhong Lu, Litao Li, Yaxin Chen, Xufei Li, Jiaying Wang, and Wei Yao. Multi-strip stitching imaging mission planning method for sar satellites regional mapping considering on-board energy consumption. *IEEE Transactions on Geoscience and Remote Sensing*, 2024.
- [12] Wei Yao, Xin Shen, Guo Zhang, Zezhong Lu, Jiaying Wang, Yanjie Song, and Zhiwei Li. A spiking neural network based proximal policy optimization method for multi-point imaging mission scheduling of earth observation satellite. *Swarm and Evolutionary Computation*, 94:101867, 2025.
- [13] Volodymyr Mnih, Koray Kavukcuoglu, David Silver, Andrei A. Rusu, Joel Veness, Marc G. Bellemare, Alex Graves, Martin Riedmiller, Andreas K. Fidjeland, Georg Ostrovski, Stig Petersen, Charles Beattie, Amir Sadik, Ioannis Antonoglou, Helen King, Dhharshan Kumaran, Daan Wierstra, Shane Legg, and Demis Hassabis. Human-level control through deep reinforcement learning. *Nature*, 518:529–533, 2015.
- [14] Christopher J. C. H. Watkins and Peter Dayan. Q-learning. *Machine Learning*, 8:279–292, 1992.
- [15] Richard S. Sutton. Dyna, an integrated architecture for learning, planning, and reacting. *ACM SIGART Bulletin*, 2(4):160–163, 1991.
- [16] Shixiang Gu, Timothy Lillicrap, Ilya Sutskever, and Sergey Levine. Continuous deep q-learning with model-based acceleration. In *Proceedings of the 33rd International Conference on Machine Learning*, volume 48 of *Proceedings of Machine Learning Research*, pages 2829–2838, 2016.
- [17] Andrew Forney, Judea Pearl, and Elias Bareinboim. Counterfactual data-fusion for online reinforcement learners. In *Proceedings of the 34th International Conference on Machine Learning*, volume 70 of *Proceedings of Machine Learning Research*, pages 1156–1164, 2017.
- [18] Núria Armengol Urpí, Marco Bagatella, Marin Vlastelica, and Georg Martius. Causal action influence aware counterfactual data augmentation. In *Proceedings of the 41st International Conference on Machine Learning*, volume 235 of *Proceedings of Machine Learning Research*, pages 1709–1729, 2024.

- [19] Thomas Mesnard, Theophane Weber, Fabio Viola, Shantanu Thakoor, Alaa Saade, Anna Harutyunyan, Will Dabney, Thomas S. Stepleton, Nicolas Heess, Arthur Guez, Eric Moulines, Marcus Hutter, Lars Buesing, and Remi Munos. Counterfactual credit assignment in model-free reinforcement learning. In *Proceedings of the 38th International Conference on Machine Learning*, volume 139 of *Proceedings of Machine Learning Research*, pages 7654–7664, 2021.
- [20] Ege Can Kaya, Mahsa Ghasemi, and Abolfazl Hashemi. Joint mdps and reinforcement learning in coupled-dynamics environments. In *Proceedings of the 42nd Conference on Uncertainty in Artificial Intelligence*, volume 337 of *Proceedings of Machine Learning Research*, pages 2877–2893, 2026.
- [21] Marc G. Bellemare, Georg Ostrovski, Arthur Guez, Philip S. Thomas, and Rémi Munos. Increasing the action gap: New operators for reinforcement learning. In *Proceedings of the Thirtieth AAAI Conference on Artificial Intelligence*, pages 1476–1483, 2016.
- [22] NASA Small Spacecraft Systems Virtual Institute. State-of-the-art of small spacecraft technology: Small spacecraft avionics. <https://www.nasa.gov/smallsat-institute/sst-soa/small-spacecraft-avionics/>, 2026. Accessed 26 August 2026.
- [23] NASA Small Spacecraft Systems Virtual Institute. State-of-the-art of small spacecraft technology: Ground data systems and mission operations. <https://www.nasa.gov/smallsat-institute/sst-soa/ground-data-systems-and-mission-operations/>, 2026. Accessed 26 August 2026.
- [24] Consultative Committee for Space Data Systems. Image data compression. Technical Report CCSDS 122.0-B-2, CCSDS, September 2017.
- [25] Hado van Hasselt, Arthur Guez, and David Silver. Deep reinforcement learning with double q-learning. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 30, pages 2094–2100, 2016.
- [26] NASA Small Spacecraft Systems Virtual Institute. State-of-the-art of small spacecraft technology. <https://www.nasa.gov/smallsat-institute/sst-soa/>, 2026. Accessed 26 August 2026.